

SkillStreak AI — Complete Project Reference

A single-file, end-to-end reference for the SkillStreak AI codebase: what it is, why it exists, every system that makes it up, every piece of tech in play, and how to reason about the code without getting lost. Written so a future engineer (or LLM context window) can pick it up cold.

Date: 2026-05-14 **Author:** Likith (with Claude) **Companion docs:** `pLanning/SKILLSTREAK_AI_PLANNING.md` (the original design doc) and `pLanning/BATTLEPLAN.md` (the phase-by-phase plan).

0. At a glance

- **Product:** SkillStreak AI — an AI-powered tech learning platform.
- **Type:** Final-year B.E. project; one-semester scope (14 weeks).
- **State (May 2026):** Phases 0–10 complete. Phase 11 (deploy + report) is the remaining work.
- **Stack one-liner:** Next.js 14 App-Router + Express + Prisma on Supabase (Postgres + pgvector) + Gemini (LLM), with Redis-or-in-memory cache.
- **Domains live:** 9 curated, ~60 topics each → ~540 topics seeded.
- **Tests:** 56 backend unit/integration tests passing.
- **Repo:** monorepo (`frontend/` , `backend/` , `planning/`) with npm workspaces. Single `npm run dev` boots both apps via `concurrently` .

1. What the project is

SkillStreak AI helps a learner master a tech domain end-to-end:

1. They pick a domain (e.g. Cybersecurity, Machine Learning).
2. They follow a curated roadmap of ~60 topics in three phases (foundations → core → advanced).
3. Each topic shows an **AI-generated explainer** (600–1000 words, sectioned), curated **YouTube videos + docs**, and an **AI-generated 5-question MCQ quiz**.
4. A **streaming AI chatbot** is always one click away — topic-aware, level-aware, with intent classification (explain / roadmap / quiz / recommend / doubt / motivational).
5. **Streaks, XP, levels, badges, leaderboard** keep the learner showing up daily.
6. **Admin panel** lets an operator edit topics, see metrics, fire test emails, and dispatch streak/digest notifications.

The "AI" in the name is more than a chat wrapper: every topic explanation, every quiz, every roadmap can be LLM-generated and cached; topic recommendations use **embeddings** (pgvector cosine similarity) over the learner's completed-topic mean vector.

Who it's for

Persona	Need
Engineering students	Structured prep across domains
Career switchers	Clear roadmap from zero to job-ready
Self-learners	Friendly, paced, motivating learning
Interview aspirants	Theory revision with quizzes

Problem it solves

Self-learners have content (Coursera, YouTube, blogs) but no direction. ChatGPT has direction-on-demand but no structure, no progress, no accountability. SkillStreak fuses **curation + AI tutor + gamification** into one accountable daily loop.

2. Goals (what we set out to build)

Primary

- A full-stack web app with personalised tech learning.
- Genuine LLM integration (not a chat wrapper): explanations, quizzes, roadmaps, intent routing, embeddings-based recommendations.
- Roadmaps for nine domains.
- Progress tracking with adaptive difficulty signals.
- Gamification mechanics that drive consistency.

Secondary

- Demonstrate end-to-end SWE: requirements → design → impl → testing → deployment.
- Showcase a defensible non-chatbot AI component (embedding recommender).
- Produce docs an examiner can read: architecture diagram, ER diagram, API spec, test plan, security review.

Explicit non-goals (out of scope for the MVP)

- Native mobile apps. (Web only, but responsive.)
 - Sandboxed code execution / live coding labs.
 - Payments / subscriptions.
 - Live human tutoring.
 - Multi-language UI (English only).
 - Social features beyond a leaderboard.
-

3. The nine domains

All nine are now **first-class curated** roadmaps (~60 topics each, 540 total topics seeded as of 2026-05-14):

#	Domain	Slug	Status
1	Cybersecurity	cybersecurity	Curated · 60 topics
2	Web Development	web-development	Curated · 59 topics
3	Artificial Intelligence	artificial-intelligence	Curated · 60 topics
4	Machine Learning	machine-learning	Curated · 60 topics
5	Data Science	data-science	Curated · 60 topics
6	Cloud Computing	cloud-computing	Curated · 61 topics
7	DevOps	devops	Curated · 60 topics
8	Blockchain	blockchain	Curated · 60 topics
9	Internet of Things	iot	Curated · 60 topics

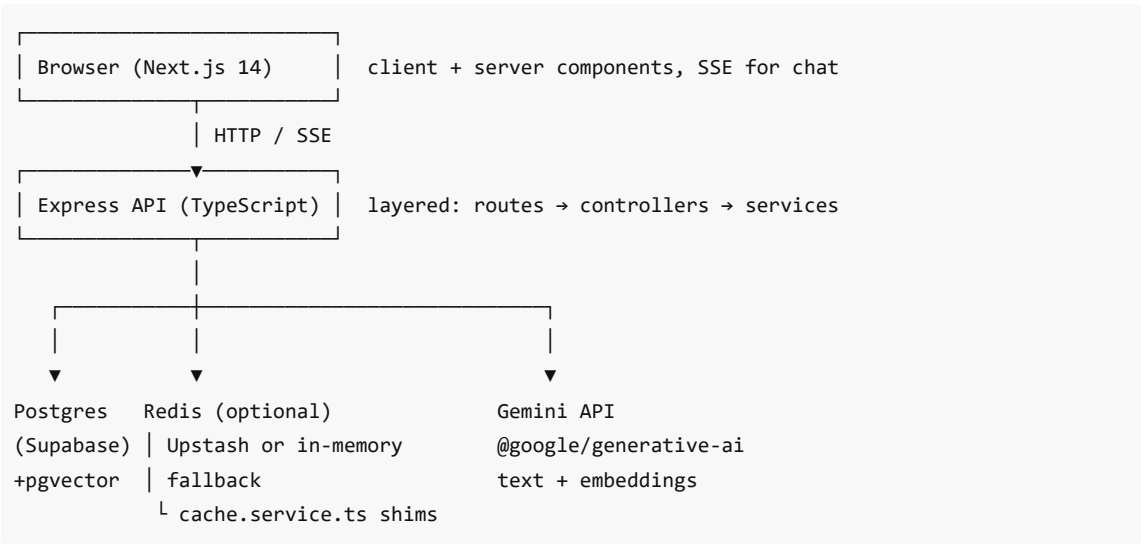
Per the original BATTLEPLAN, only the first five were planned as curated and the last four as chatbot-served on-demand. We flipped the last four to curated in commit `daa5a9d` so the experience is uniform.

Each roadmap is split into **three phases** that map to the `Topic.phase` enum:

Phase	Days	Purpose
foundations	1–30	Vocabulary, mental models
core	31–60	Main subject matter
advanced	61–90	Edge cases, current trends

4. Architecture

4.1 Top-level shape



External services used:

- **Gemini** (`@google/generative-ai`) — text completion and embeddings.
- **YouTube Data API v3** — optional, used to validate video URLs (LLM currently proposes the URLs; we extract the 11-char ID and build the thumbnail link directly).
- **Resend** (`resend`) — transactional email for streak reminders + weekly digest. No-key fallback is a quiet no-op so dev works without an API key.
- **Supabase** — managed Postgres in `ap-northeast-1` (Tokyo). pgvector extension enabled for embeddings.

4.2 Architecture style

- **Layered monolith.** Express server with route → controller → service separation. No microservices.
- **Stateless API.** JWT auth via `Authorization: Bearer ...` headers; no server-side sessions. Horizontally scalable.
- **External AI is a managed dependency.** `llm.service.ts` wraps the Gemini SDK and is the single seam every service uses for LLM calls.

4.3 Request lifecycle (worked example)

User asks the chatbot "Explain the CIA triad":

1. UI POSTs `/api/chat/sessions/:id/messages` with the message.
 2. `requireAuth` middleware (`auth.middleware.ts`) validates the JWT and attaches `req.user` .
 3. `validate(sendMessageSchema)` runs Zod validation on the body.
 4. The rate limiter caps the chat-send route at 20/min.
 5. `chat.controller.ts` calls `chat.service.streamMessage()` .
 6. Service loads session + topic + recent history from Postgres.
 7. `classifyIntent(message)` runs — fast heuristic regex first, LLM fallback for ambiguous cases.
 8. `buildSystemPrompt(ctx)` plus intent-specific instructions go to `llm.service.streamComplete()` .
 9. Tokens stream back; the controller writes SSE frames.
 10. On completion, the assistant message is persisted to `chat_messages` ; follow-up suggestions are generated best-effort with a separate LLM call.
-

5. Tech stack (with actual installed versions)

5.1 Backend (`backend/package.json`)

Layer	Library	Version
HTTP framework	<code>express</code>	<code>^4.21.1</code>
Type system	<code>typescript</code>	<code>^5.6.3</code>
Runtime (dev)	<code>tsx</code>	<code>^4.19.2</code> (watch mode)
ORM	<code>prisma</code> + <code>@prisma/client</code>	<code>^5.22.0</code>
LLM SDK	<code>@google/generative-ai</code>	<code>^0.24.1</code>
Auth — JWT	<code>jsonwebtoken</code>	<code>^9.0.2</code>
Auth — hashing	<code>bcryptjs</code>	<code>^2.4.3</code>
Validation	<code>zod</code>	<code>^3.23.8</code>
Security headers	<code>helmet</code>	<code>^8.0.0</code>
CORS	<code>cors</code>	<code>^2.8.5</code>
Rate limit	<code>express-rate-limit</code>	<code>^7.4.1</code>
Request logging	<code>morgan</code>	<code>^1.10.0</code>
Cache (when on)	<code>ioredis</code>	<code>^5.4.1</code>
Supabase client	<code>@supabase/supabase-js</code> , <code>@supabase/ssr</code>	<code>^2.105.1</code> , <code>^0.10.2</code>
Email	<code>resend</code>	<code>^6.12.2</code>
Env loader	<code>dotenv</code>	<code>^16.4.5</code>
Tests	<code>jest</code> , <code>ts-jest</code> , <code>supertest</code>	<code>^29.7.0</code> , <code>^29.2.5</code> , <code>^7.0.0</code>

5.2 Frontend (`frontend/package.json`)

Layer	Library	Version
-------	---------	---------

Framework	next	14.2.18 (App Router)
React	react, react-dom	^18.3.1
TypeScript	typescript	^5.6.3
Styling	tailwindcss	^3.4.14
Components	shadcn/ui-style primitives + @radix-ui/*	various
Animations	framer-motion	^12.38.0
3D / R3F	@react-three/fiber, @react-three/drei, three	^8.18.0, ^9.122.0, ^0.160.1
3D scene	@splinetool/react-spline, @splinetool/runtime	^4.1.0, ^1.12.90
Icons	lucide-react	^0.454.0
Charts	recharts	^3.8.1
Markdown	react-markdown + remark-gfm	^10.1.0, ^4.0.1
Forms	react-hook-form + @hookform/resolvers + zod	^7.75.0
Toasts	sonner	^2.0.7
Confetti	canvas-confetti	^1.9.4
State (client)	zustand	^5.0.12
HTTP	axios	^1.7.7
Tailwind plugins	@tailwindcss/typography, tailwindcss-animate	^0.5.19, ^1.0.7

5.3 Local-dev orchestration

Root `package.json` declares npm workspaces (`frontend` , `backend`) and this script:

```
"dev": "concurrently -n frontend,backend -c cyan,magenta \"npm:dev:frontend\"
\"npm:dev:backend\""
```

So `npm run dev` from repo root brings up the Next.js app on `http://localhost:3000` and the Express API on `http://localhost:4000` in parallel, with colour-tagged interleaved logs.

6. Data model (Prisma schema)

The actual schema lives in `backend/prisma/schema.prisma` and is migrated into Supabase. The shapes below summarise it; field names match the code.

6.1 Identity

```
model User {
  id          String      @id @default(cuid())
  email       String      @unique
```

```

passwordHash String
name          String
role          String      @default("user") // "user" | "admin"
level         Int         @default(1)
xp            Int         @default(0)
createdAt     DateTime    @default(now())
updatedAt     DateTime    @updatedAt
profile       UserProfile?
progress      UserProgress[]
quizAttempts  QuizAttempt[]
chatSessions  ChatSession[]
badges        UserBadge[]
streak        Streak?
xpEvents      XpEvent[]
}

model UserProfile {
  userId      String  @id
  priorLevel  String  // none | some | experienced
  goal        String  // interview | awareness | curiosity
  pace        String  // relaxed | standard | intense
  preferredDomains String[]
  reminderTime String? // HH:mm
  digestEnabled Boolean @default(true)
  onboardedAt DateTime @default(now())
}

```

6.2 Content

```

model Domain {
  id      String  @id @default(cuid())
  slug    String  @unique
  name    String
  description String
  color   String
  icon    String
  difficulty String
  isCurated Boolean
  orderIndex Int
  roadmaps Roadmap[]
}

model Roadmap {
  id      String  @id @default(cuid())
  domainId String
  title   String
  totalTopics Int
  estimatedDays Int
  topics  Topic[]
}

```

```

model Topic {
  id          String   @id @default(cuid())
  roadmapId   String
  orderIndex  Int
  title       String
  summary     String
  difficulty  String   // easy | standard | hard
  phase       String   // foundations | core | advanced
  embedding   Unsupported("vector(1536)")? // pgvector
  progress    UserProgress[]
  attempts    QuizAttempt[]
  resources   Resource[]
  chats       ChatSession[]
  @@unique([roadmapId, orderIndex])
}

model Resource {
  id          String   @id @default(cuid())
  topicId     String
  type        String   // video | doc
  title       String
  url         String
  source      String
  thumbnailUrl String?
  durationSeconds Int?
  orderIndex  Int
  qualityScore Float    @default(0)
  createdAt   DateTime @default(now())
}

```

6.3 Progress + gamification

```

model UserProgress {
  id          String   @id @default(cuid())
  userId      String
  topicId     String
  status      String   // not_started | in_progress | completed
  bestScore   Int?
  timeSpentSeconds Int    @default(0)
  completedAt DateTime?
  @@unique([userId, topicId])
}

model QuizAttempt {
  id          String   @id @default(cuid())
  userId      String
  topicId     String
  level       String
  questions   Json
  userAnswers Json
  score       Int
}

```

```

    total      Int
    passed      Boolean
    attemptedAt DateTime @default(now())
}

model Streak {
    userId      String    @id
    currentStreak Int      @default(0)
    longestStreak Int      @default(0)
    lastActiveDate DateTime?
    freezesAvailable Int    @default(1)
    freezesUsedAt DateTime?
}

model Badge {
    id          String    @id @default(cuid())
    slug        String    @unique
    name        String
    description String
    icon        String
    criteria    Json
    orderIndex  Int
}

model UserBadge {
    userId      String
    badgeId     String
    earnedAt    DateTime @default(now())
    @@id([userId, badgeId])
}

model XpEvent {
    id          String    @id @default(cuid())
    userId      String
    amount      Int
    reason      String    // topic_complete | first_topic_of_day | quiz_pass | quiz_perfect |
    streak_day | comeback
    meta        Json?
    createdAt   DateTime @default(now())
}

```

6.4 Chat

```

model ChatSession {
    id          String    @id @default(cuid())
    userId      String
    topicId     String?
    title       String
    createdAt   DateTime   @default(now())
    updatedAt   DateTime   @updatedAt
    messages    ChatMessage[]
}

```



```

}

model ChatMessage {
  id          String   @id @default(cuid())
  sessionId   String
  role        String   // user | assistant | system
  content     String
  intent      String?
  createdAt   DateTime @default(now())
}

```

6.5 Indexes that matter

- `UserProgress (userId, topicId)` — uniqueness + lookups.
- `ChatMessage (sessionId, createdAt)` — history pagination.
- `QuizAttempt (userId, topicId, attemptedAt DESC)` — history.
- `topics USING ivfflat (embedding vector_cosine_ops)` — recommender.

7. API surface (one line per endpoint)

Mount root: `/api` . Routes registered in `backend/src/routes/index.ts` .

7.1 Health

- `GET /api/health` — pings DB; returns `{status, checks: {database, redis}}` .

7.2 Auth (`/api/auth`)

- `POST /register` — create user, returns `{user, token}` .
- `POST /login` — returns `{user, token}` on success, 401 otherwise.
- `GET /me` — current user with profile.
- `PATCH /me/profile` — upsert onboarding profile.
- `PATCH /me/settings` — partial update (reminderTime, digestEnabled).
- `POST /logout` — stateless no-op.

7.3 Domains + roadmaps

- `GET /api/domains` — list with progress per domain.
- `GET /api/domains/:slug` — one domain.
- `GET /api/roadmap/:domainSlug` — curated roadmap + topic statuses.

7.4 Topics (`/api/topics`)

- `GET /:id` — topic detail + user progress on it.
- `POST /:id/explain` — LLM long-form explainer (cached 30 days).
- `POST /:id/complete` — mark complete; fires XP/streak/badge updates.

7.5 Chat (`/api/chat`)

- `GET /sessions` — list user sessions.
- `POST /sessions` — create a session (optional topicId).
- `GET /sessions/:id` — full session with messages.
- `PATCH /sessions/:id` — rename.
- `DELETE /sessions/:id` — delete.

- `POST /sessions/:id/messages` — send message; **SSE** stream.

7.6 Quiz (`/api/quiz`)

- `GET /history` — last 50 attempts.
- `POST /topics/:id/generate` — generate (or retrieve cached) 5-MCQ quiz.
- `POST /topics/:id/submit` — score and persist; awards XP/streak.

7.7 Progress / analytics (`/api/progress`)

- `GET /` — overall summary.
- `GET /heatmap` — 90-day activity heatmap data.
- `GET /weak-areas` — topics with low best-scores.
- `GET /score-trend` — recent quiz scores series.

7.8 Gamification (mounted at `/api/`)

- `GET /streak` — current streak + freezes.
- `POST /streak/freeze` — spend a freeze.
- `GET /badges` — all + earned timestamps.
- `GET /leaderboard` — weekly XP top 10.

7.9 Resources

- `GET /api/resources/:topicId` — curated videos + docs (cached 7 days).

7.10 Admin (`/api/admin` , role-gated)

- `GET /topics` — list topics for editing.
- `PATCH /topics/:id` — edit.
- `DELETE /topics/:id` — delete.
- `GET /metrics` — platform metrics.
- `GET /email/status` — Resend key present?
- `POST /email/test/streak` — send test reminder.
- `POST /email/test/digest` — send test digest.
- `POST /email/dispatch/streaks` — fan-out streak emails.
- `POST /email/dispatch/digests` — fan-out digest emails.

8. Auth & security

8.1 Auth model

- **JWT** signed with `JWT_SECRET` , default expiry `7d` .
- Token issued on `/register` and `/login` ; client stores in `localStorage` under `skillstreak.token` and attaches as `Authorization: Bearer ...` via an axios request interceptor.
- A response interceptor clears the token on a 401 *on protected routes* but **not** on a failed `/auth/login` (user never had a valid token).

8.2 Password hashing

- `bcryptjs` with `env.BCRYPT_ROUNDS` (default 10). Hash format validated in tests (`$2[aby]$10$...`).

8.3 Middleware chain

- `requireAuth` — JWT signature + payload shape check; throws 401.
- `requireAdmin` — gates `/admin/*` . Combined with `requireAuth` at the router level.

- `validate(schema, source)` — Zod validation on every write/read.
- `errorHandler` — last in chain; surfaces `ApiError` as JSON.

8.4 Rate limits

Endpoint	Limit	Window
Global	120 req	1 min
<code>/auth/register, /auth/login</code>	30	15 min
<code>/chat/sessions/:id/messages</code>	20	1 min
<code>/quiz/topics/:id/generate</code>	10	1 min
<code>/topics/:id/explain</code>	30	1 min

Note: `express-rate-limit` is in-memory, so multi-instance deployments need `rate-limit-redis`.
 Documented in `SECURITY_REVIEW.md`.

8.5 Other

- **Helmet** for security headers.
- **CORS** restricted to `env.FRONTEND_ORIGIN` only, with credentials.
- **Prompt-injection sanitizer** (`utils/sanitize.util.ts`) blocks the obvious "ignore previous instructions" family and caps messages at 2000 chars. The regex was widened during Phase 10 testing — see commit `e80006d`.
- **XSS**: chat + topic markdown rendered via `react-markdown` without `rehype-raw` and without `allowDangerousHtml`, so raw HTML is escaped.
- **CSRF**: N/A — auth is header-based, not cookie-based.
- **SQL injection**: Prisma covers 99%; one `recommender.service.ts` raw-SQL is parameterised, with the only inline interpolation being DB-issued cuids (low risk, documented for hardening).

8.6 Phase 10 security review

Full audit in `planning/SECURITY_REVIEW.md`. Every battleplan §10 checklist item is **PASS**. Two LOW findings deferred to Phase 11 (in-memory rate limiter, optional `Prisma.join` refactor in the recommender).

9. AI chatbot (the centrepiece)

9.1 What it does

Streaming, intent-routed, topic-aware tutor that supports six intents:

Intent	Trigger phrasings	Behaviour
<code>explain</code>	"what is ...", "explain ..."	Definition + structured prose.
<code>roadmap</code>	"roadmap for ...", "where do I start"	Custom roadmap JSON.
<code>quiz</code>	"quiz me", "MCQ on ..."	5 MCQs in JSON.
<code>recommend</code>	"videos on ...", "resources"	Ranked resource list.
<code>doubt</code>	"error", "stuck", "why doesn't this work"	Debug-style answer.

motivational	"burnt out", "giving up"	Empathetic + actionable.
--------------	--------------------------	--------------------------

9.2 Intent routing

File: `backend/src/prompts/intent.prompt.ts` .

- **Cheap regex heuristics first** (deterministic, free). The `motivational` regex was patched (`motivat` → `motivat\w*`) in Phase 10 testing so "I'm losing motivation" now matches.
- **LLM fallback** with `temperature=0` and an 8-token cap for ambiguous inputs. Output forced to one of the six labels.
- Default on uncertainty: `explain` .

9.3 System prompt

File: `backend/src/prompts/system.prompt.ts` . Injects user level (beginner / intermediate / advanced — mapped from `priorLevel`), the current topic and domain, and behaviour rules (one correct framing, plain language unless advanced, end with a follow-up nudge).

9.4 Streaming

SSE over `POST /chat/sessions/:id/messages` . Frames yielded:

```
{ type: "intent",  intent }      // once, after classification
{ type: "delta",   text }        // many, one per token chunk
{ type: "done",    messageId, followUps: string[] } // once
```

Client (`lib/api.ts` `streamMessage`) parses these and forwards into the `ChatStore`. The frontend renders `react-markdown` with `remark-gfm` and syntax-highlighted code blocks.

9.5 Persistence + history

Every user + assistant turn is written to `chat_messages` . On every stream the service loads the **last 10 turns** as `ChatTurn[]` history. Sessions auto-title from the first user message (truncated to 60 chars). Sidebar lists by `updatedAt` .

9.6 Caching + cost control

- **30-day cache** on topic explanations (`topic:{id}:level:{lv1}:v2`). The `v2` suffix was added when the explainer prompt was expanded from 150–250 words to 600–1000 words in commit `d283ca4` ; older cached entries fell out automatically.
- **7-day cache** on resources.
- **No cache** on free-form chat — every chat is bespoke to the conversation.

9.7 Sanitisation

`sanitizeChatInput` rejects empty/too-long messages and the seven injection-pattern regexes. Tests cover ordinary tech questions and "act as a tutor" allow-list.

10. Quiz engine

File: `backend/src/services/quiz.service.ts` .

- **Generation:** LLM is asked for exactly 5 MCQs, each with 4 options and a `correctIndex 0..3` , plus an explanation. Output forced to JSON with a strict schema; `parseQuestions` rejects malformed shapes with

a 502 ApiError.

- **Validator pass:** every generated question is run through a second LLM call (`QUIZ_VALIDATOR_SYSTEM`) that returns `{valid:true|false, reason?}` . Up to **two attempts** to generate a valid set; if both flag failures, we accept the second batch rather than fail the user.
- **Caching:** quizzes are cached for **7 days** per topic + level. A `?fresh=1` retake bypasses the cache.
- **Scoring:** server compares user answers to `correctIndex` , computes score, returns per-question breakdown.
- **Pass threshold: 3/5** (60%) marks the topic complete; less leaves it `in_progress` .
- **Idempotent completion:** quiz pass and the explicit "Mark complete" button both award the right XP without double-counting.

11. Progress & analytics

File: `backend/src/services/progress.service.ts` .

Surfaces consumed by the dashboard:

- **Overall summary** — topics completed / total, current streak, level, XP, weekly XP.
- **Per-domain progress** — completed / total per domain.
- **90-day heatmap** — daily activity counts.
- **Weak areas** — topics where `bestScore < 3` on the latest attempt.
- **Score trend** — last N attempts series, charted with Recharts.

12. Gamification

File: `backend/src/services/gamification.service.ts` .

12.1 XP rules

Action	XP
Topic completed	+10
First topic of the day	+10
Quiz passed (3/5+)	+15
Perfect quiz (5/5)	+25
Streak day maintained	+5
Comeback (first action after a break)	+50

12.2 Level math

`levelForXp(xp)` finds the largest `level` such that $100 * level * \log_2(level+1) \leq xp$. The function is exported and covered by tests in `tests/services/level.test.ts` (monotonic, boundary check, reconstructibility).

12.3 Streak logic

- Bumped lazily inside `recordActivity()` rather than via cron.
- Day boundaries are **UTC**.

- Freezes: 1 banked at signup, refills to 1 every **Monday** (max 2 banked).
- A streak break awards a **+50 comeback bonus** on the next qualifying action — explicitly anti-shame.
- Helpers (`utcDateOnly` , `daysBetweenUtc` , `isMondayUtc` , `startOfIsoWeekUtc`) are exported and covered by tests.

12.4 Badges (8)

Slug	Earned when
first-step	First topic complete
week-warrior	7-day current/longest streak
marathoner	30-day current/longest streak
quiz-master	10 perfect quizzes
domain-pioneer	First domain roadmap fully done
polyglot	Topics started in 5+ domains
insomniac	Topic completed 00:00–04:00
early-bird	Topic completed 04:00–07:00

Badges are idempotent: `evaluateBadges()` upserts via `userBadge.create` catching the unique-violation race.

12.5 Leaderboard

Top 10 by sum of `XpEvent.amount` for the current ISO week (Mon UTC → now). Returns rank, user, level, total XP, week XP.

13. Resources

File: `backend/src/services/resource.service.ts` .

- An LLM "librarian" returns 4 videos + 3 docs per topic.
- For each video the LLM proposes a `videoUrl` and a `searchQuery` .
- We parse the URL and accept it if it matches `youtube.com/watch?v=ID` , `youtu.be/ID` , `/embed/ID` , or `/shorts/ID` with an 11-char id — validated against `^[A-Za-z0-9_-]{11}$` .
- Valid → we link directly to the video and set the thumbnail to `i.ytimg.com/vi/<id>/hqdefault.jpg` .
- Invalid / blank → we fall back to a canonical `youtube.com/results?search_query=...` link (no fabricated IDs).
- For docs, the LLM gives `{title, source, url}` from a reputable list (MDN, OWASP, web.dev, fast.ai, scikit-learn, Mozilla, etc.). URLs are validated as http/https.
- Cached per topic for 7 days. `scripts/clear-resources.ts` wipes the cache on demand.

13.1 Embedding-based recommender

File: `backend/src/services/recommender.service.ts` .

- Each topic gets a 1536-dim embedding from Gemini's embedding model on first request (lazy backfill, capped at 8 per call to keep latency predictable).
- User "interest" embedding = mean of completed-topic embeddings.
- Top-K by `1 - cosine_distance` (`pgvector <=>`) excluding completed topics.

- Cold start (no completed topics) falls back to the first `foundations` -phase topic in the user's preferred domains.

14. Notifications & admin

14.1 Email

- `email.service.ts` wraps `resend` and falls back to a no-op when `RESEND_API_KEY` is unset (so dev never fails).
- Two campaigns: **streak reminder** (HH:mm per user preference, if streak would lapse) and **weekly digest** (Sunday).
- The cron was deferred to Phase 11 (deployment). For demo, admin has manual **fire-now** buttons.

14.2 Admin panel

Role-gated UI at `/admin`. Surfaces:

- Topic list + inline edit + delete.
- Platform metrics (users, sessions, topics, quizzes, recent activity).
- Email status badge (`RESEND_API_KEY` present?).
- Manual triggers: send-test reminder/digest, dispatch reminders to all eligible, dispatch digests.

Promotion path: `UPDATE users SET role='admin' WHERE email='...'` and log out / log back in so the JWT is reissued with the new role.

15. Frontend UX surfaces

15.1 Route map (App Router, route groups)

```
app/
├ layout.tsx                // root: <html dark>, Inter font, Sonner Toaster
├ page.tsx                  // landing
├ not-found.tsx             // custom 404
├ (auth)/
│ └ layout.tsx              // split-pane: animated chars + form panel
│   └ auth-form-context.tsx // typing/showPassword state for the panel
│     └ login/page.tsx      // RHF + Zod, shake, caps lock, hints
│       └ register/page.tsx // + live password strength meter
├ onboarding/page.tsx       // 3-question survey + domain preferences
└ (dashboard)/
  └ layout.tsx              // useBootstrapUser, AnimatePresence, skip-link, ChatWidget
    └ dashboard/page.tsx    // streak banner, today's topic, heatmap, charts
      └ domains/page.tsx    // domain gallery + 3D Whobee robot
        └ domains/[slug]/page.tsx // roadmap winding trail
          └ topic/[id]/page.tsx // AI explainer + sidebar (mark/quiz/resources/chat)
            └ quiz/[topicId]/page.tsx // 5-question MCQ player + result + confetti
              └ quiz/[topicId]/loading.tsx // "Generating quiz..." PageLoader
                └ chatbot/page.tsx // full-screen chat + session sidebar (hidden on mobile)
                  └ progress/page.tsx // analytics expanded
                    └ leaderboard/page.tsx // weekly top 10
                      └ settings/page.tsx // account / streak rescue / preferences / notifications /
```

```
sign-out
└─ admin/page.tsx // admin-gated content + metrics + email triggers
```

15.2 Landing page sections

1. **Hero** with Spline 3D "Whobee" robot, headline, scroll prompt.
2. **Domain marquee** — auto-scrolling ticker of all 9 domains.
3. **Stats banner** — 4 counters that count up when in view (9 domains, 90+ topics, 5 MCQs/topic, 8 badges).
4. **BurstSection** — wordmark pops + radial burst from the "a", time-based animation triggered at 55% in-view (`useInView + once:true`).
5. **HowItWorks** — 3 step cards: pick → learn → earn.
6. **Bento features grid** — 5 cards (AI tutor, roadmaps, quizzes, streaks, analytics) — the AI tutor card spans 2x2.
7. **FinalCTA** — closing card with the shared `HeroCTA` .
8. **Footer**.

15.3 Shared components worth knowing

- `components/shared/Navbar.tsx` — sticky top bar, level/XP pill, Domains, Leaderboard, Chat, Admin (role-gated), Settings, Log out.
- `components/shared/PageLoader.tsx` — reusable spinner with optional hint.
- `components/chat/ChatWidget.tsx` — floating bottom-right button (hidden on the dedicated chatbot page).
- `components/landing/HeroCTA.tsx` — the unified primary + secondary CTA.
- `components/3d/SplineScene.tsx` — Spline wrapper that **pauses the WebGL loop when scrolled out of view** (single biggest landing perf win).

15.4 Animations stack

- **framer-motion** — page transitions, scroll reveals, button micro- interactions, error shake (now CSS), gamification toast slides.
- **CSS keyframes** — aurora bg, marquee, button breathe, form shake, loader spin.
- **canvas-confetti** — perfect-quiz celebration.
- **prefers-reduced-motion** is honoured everywhere via `useReducedMotion()` .

15.5 Auth UX highlights (a lot of polish lives here)

- `react-hook-form` + Zod resolver with `mode: "all"` + `reValidateMode: "onChange"` so `isValid` is always live → the submit button is physically disabled while any field is invalid.
- **Inline FieldError** with AlertCircle icon when Zod rejects a field.
- **Red input border** the moment a field is invalid (not just on backend rejection).
- **Below-button hint**: "Enter a valid email address to continue", "Password needs at least 8 characters" — clears the moment the form is valid.
- **CSS shake** retrIGGERED via `key={form-#{errorBumpKey}}` on bad submit.
- **Caps Lock detector** — keyboard event listener flips an amber warning when the password field has content and Caps Lock is on.
- **Password auto-clear + refocus** after a wrong login.
- **"Forgot password?"** link appears next to the password label after the first failed attempt.
- **Live password strength meter** on register: 4 segments + 4 named rules (8+ chars, letters+numbers, upper+lower, a symbol).

16. Frontend perf (Phase 9 polish)

After Phase 9 we hit a wall where the landing felt laggy. Fixes that held:

- **Spline pause on scroll-out** — the 3D scene's WebGL render loop was running forever. Now we use `IntersectionObserver` and call `app.stop()` / `app.play()` based on visibility. Biggest single win.
- **Aurora bg** — reduced `filter: blur(120px)` → `blur(80px)` and `60vmax` → `45vmax`; added `contain: paint` + `transform: translate3d(0,0,0)` so each blob is its own GPU layer.
- **BurstSection circle** — was `200vmax` with a `200px` box-shadow; now `140vmax` with no shadow, `contain: paint`.
- **backdrop-blur audit** — most non-essential `backdrop-blur` calls were downgraded to `-sm` or removed.

Result: scrolling went from jittery to smooth on a mid-range laptop.

17. Testing (Phase 10)

Stack: Jest + ts-jest + a `setup-env.ts` that loads safe defaults for `NODE_ENV=test`, `JWT_SECRET`, `BCRYPT_ROUNDS=4` (fast hashing for tests), etc.

56 tests, 7 suites, all passing.

Suite	Coverage
tests/utils/password.test.ts	hash round-trip, bcrypt prefix, salt randomness
tests/utils/jwt.test.ts	sign/verify, tampering, foreign secret, expiry
tests/utils/sanitize.test.ts	char cap, empty, injection patterns, allow-list
tests/services/level.test.ts	monotonicity, boundary, reconstructibility
tests/services/streakDates.test.ts	UTC day math, Monday detection, ISO week start
tests/services/intent.test.ts	15+ phrasings → correct intent
tests/services/quizPrompt.test.ts	prompt builder snapshots, system contract

Bugs found and fixed during the test pass (recorded in `planning/bugs.md`):

1. `sanitize.util` injection regex was too narrow — "ignore all previous instructions" slipped past. Widened to accept chained qualifiers.
2. `intent.prompt` motivational regex `\bmotivat\b` couldn't match "motivation". Changed to `motivat\w*`.

Plus a **manual QA checklist** in `planning/QA_CHECKLIST.md` and the **security audit** in `planning/SECURITY_REVIEW.md`.

18. Local dev workflow

18.1 Environment

Two `.env` files (each has an `.env.example` checked in):

`backend/.env` (essentials):

```

NODE_ENV=development
PORT=4000
FRONTEND_ORIGIN=http://localhost:3000
DATABASE_URL=postgresql://postgres.<project>:<pwd>@aws-1-ap-northeast-
1.pooler.supabase.com:5432/postgres
REDIS_URL=                # optional; empty = in-memory fallback
JWT_SECRET=<16+ chars>
JWT_EXPIRES_IN=7d
BCRYPT_ROUNDS=10
LLM_PROVIDER=gemini
GEMINI_API_KEY=<key>
LLM_MODEL=gemini-flash-lite-latest
YOUTUBE_API_KEY=          # optional
RESEND_API_KEY=           # optional; no-op fallback if blank
EMAIL_FROM=SkillStreak AI <onboarding@resend.dev>

```

frontend/.env.local :

```

NEXT_PUBLIC_API_URL=http://localhost:4000/api
NEXT_PUBLIC_SPLINE_SCENE= # optional override of the Whobee scene URL

```

18.2 Common commands

```

# Boot both servers
npm run dev

# Backend
npm --workspace backend run type-check
npm --workspace backend test
npm --workspace backend run db:seed
npx tsx backend/scripts/clear-resources.ts

# Frontend
npm --workspace frontend run type-check
npm --workspace frontend run build

```

18.3 Database migrations — the Likith-specific gotcha

`prisma migrate dev` is **interactive** on Windows and tends to hang on shadow-DB prompts when pointed at Supabase. The reliable workflow:

1. Edit `prisma/schema.prisma` .
2. `prisma migrate diff --from-schema-datasource ... --to-schema-datamodel ... --script > migrations/<n>/migration.sql`
3. Apply via the `Write` tool (CRLF gotcha: don't let an editor add a UTF-8 BOM — Prisma rejects it).
4. `prisma migrate deploy` (non-interactive).

Watch out for DLL locks on Windows if a `prisma` process is running when you swap binaries.

19. Phase timeline (the BATTLEPLAN, marked up)

#	Phase	Tagline	Status
0	Foundation Setup	Stage everything that follows	✅ done
1	Auth & User System	Real users, real sessions	✅ done
2	Domains, Roadmaps & Topics	Build the spine	✅ done (all 9 curated, ~60 each)
3	AI Chatbot (Core)	The crown jewel	✅ done
4	Quiz Engine	Make it test	✅ done
5	Progress & Analytics	Make it measurable	✅ done
6	Streaks & Gamification	Make it sticky	✅ done
7	Resource Recommender	The smart "next step"	✅ done (incl. embeddings + direct video URLs)
8	Notifications & Admin	Operator + retention	✅ done
9	Frontend Excellence	Polish, perf, accessibility	✅ done
10	Testing & Hardening	Break it before the examiner does	✅ done
11	Deployment & Demo Prep	Ship it, sell it	🕒 remaining

Commit naming convention

`<phase>-<short-name>: <what changed>` — e.g. `phase-10-testing: jest scaffold + 56 unit tests, QA checklist, security review`. See `git log --oneline` for the full chronology.

20. Conventions & gotchas

20.1 Conventions

- **Vertical slices, not horizontal.** Build one feature end-to-end (DB → API → UI) before starting the next.
- **No mocks in DB tests.** Tests that touch DB use the real Supabase test connection; mock-vs-prod divergence has burned us before.
- **Battleplan is the contract.** Phase commits are named accordingly.
- **No comments unless WHY is non-obvious.** Identifier names should tell you WHAT.
- **No backwards-compatible shims.** If something is unused after a change, delete it.
- **Validate at boundaries only.** Trust internal code; Zod at the API surface.

20.2 Gotchas

- **CRLF on Windows.** Git warns on every commit; harmless but loud. `.gitattributes` is intentionally not set so files stay LF in repo and CRLF on checkout.
- **Supabase pooled connections.** Use the `?pgbouncer=true` form for app code; raw migrations need the direct connection.

- **JWT expiry baked in.** Promoting a user to admin requires a re-login because the role is in the existing token.
 - **Resend without a key** silently no-ops; tests rely on this.
 - **react-hook-form** `mode: "all"` is required for `isValid` to be live; without it the submit button could fire on a stale state.
 - **Spline must be paused off-screen** or the GPU thermals every laptop.
 - **The Next.js dev "X errors" indicator** is dev-only; it never appears in production builds.
-

21. Memory the project keeps about its operator

(See `~/.claude/projects/.../memory/MEMORY.md`.)

- **User — Likith:** final-year B.E. student, solo dev, Windows + PowerShell, intermediate tooling literacy. Prefers terse replies and proceeds without hand-holding.
 - **Project — SkillStreak AI:** academic project; `planning/BATTLEPLAN.md` is the contract; commits use the `phase-N` prefix.
 - **Prisma workflow:** `migrate dev` is interactive on Windows — use `diff` + `Write` + `deploy` instead. Watch for BOM and DLL locks.
 - **External systems:** GitHub repo, Supabase project ref `vzynftf1dydmym1qwgtb` (Tokyo region), Gemini AI Studio.
 - **Collaboration style:** terse replies = proceed; flag irreversible actions; handle accidentally-pasted secrets cleanly.
-

22. What's left for Phase 11

1. **Production hosting:** Vercel (frontend) + Render/Fly/Railway (backend). Custom domain optional.
 2. **Sentry** wiring for both apps.
 3. **Uptime monitoring** (UptimeRobot free tier).
 4. **Production env vars:** a strong `JWT_SECRET`, `BCRYPT_ROUNDS ≥ 12`, real `RESEND_API_KEY`, `FRONTEND_ORIGIN` pointing at the deployed URL.
 5. **Cron:** streak reminders + weekly digest (move from manual admin buttons to a scheduled trigger).
 6. **Rate-limit Redis store** (`rate-limit-redis`) for multi-instance.
 7. **Recommender raw-SQL** → `Prisma.join` migration (low-risk hardening).
 8. **Final report:** synthesise this document + `SECURITY_REVIEW.md` + `QA_CHECKLIST.md` + screenshots into the academic submission.
 9. **Recorded demo:** 5-minute walkthrough.
-

23. Glossary

- **Curated** — the domain has a hand-defined roadmap of topics in `seed.ts`. As of 2026-05-14 all nine domains are curated.
- **Phase** — one of `foundations` | `core` | `advanced` on a Topic; also the BATTLEPLAN's 12-step development phases.
- **Streak** — current consecutive UTC days with any qualifying action.
- **Freeze** — a banked one-day rescue for a missed day. Refills every Monday UTC, max 2 banked.
- **XP event** — a row in `xp_events` recording why XP was awarded; also the source for the weekly leaderboard sum.
- **Intent** — one of the six chatbot classifications.

- **pgvector** — the Postgres extension that backs the recommender; enabled on the Supabase project; topics have a 1536-dim `embedding` column.

End of document.